

Memoria

Introducción

La **memoria** es un componente esencial en cualquier ordenador, ya que permite almacenar y recuperar información. A nivel de hardware, está compuesto por circuitos electrónicos, y su unidad básica de almacenamiento es el **bit**. Sin embargo, las computadoras pueden trabajar con **bytes**, y en muchos sistemas, se utilizan unidades mayores llamadas **palabras**.

- Una **palabra** es la unidad natural de datos utilizada por un procesador. La mayoría de los registros del procesador tienen tamaño de palabra y las operaciones de transferencia entre la memoria y la CPU suelen realizarse en unidades de palabra.
- En sistemas modernos, el tamaño típico de palabra es de **32 o 64 bits**.

Tiempos de acceso a memoria

La memoria se puede clasificar según la forma en que se accede a la información:

- **Memoria convencional:** Se accede directamente a través de una dirección. Se le proporciona una dirección de memoria y devuelve el contenido almacenado.
- **Memoria Asociativa o CAM:** Se accede a través de un contenido. El sistema busca si ese contenido está almacenado en algún lugar. Si lo encuentra, devuelve las direcciones correspondientes. Se utiliza en cachés y funciona de forma similar a una tabla hash.

Jerarquía de memoria

El **tiempo de acceso a memoria** es el tiempo necesario para leer o escribir datos en la memoria, desde que se coloca la dirección hasta que la CPU accede o almacena los datos. Existe un compromiso entre velocidad, capacidad y coste.

- Memorias más rápidas son más caras.
- Memorias con mayor capacidad suelen ser más lentas.

Esto lleva al diseño de una **jerarquía de memoria**, donde cada nivel tiene diferentes características de velocidad y capacidad:

1. **Registros del procesador**
2. **Memoria caché**
3. **Memoria principal**
4. **Disco duro**
5. **Medios más lentos**

Fragmentación de memoria

La fragmentación de memoria puede ocurrir en dos formas:

- **Fragmentación interna:** Ocurre cuando se asignan bloques de tamaño fijo, pero el proceso no necesita todo ese bloque, dejando espacio desperdiciado dentro del mismo bloque.
- **Fragmentación Externa:** Aparece cuando existen suficientes bloques libres en total, pero no están contiguos, por lo que no se pueden usar para satisfacer una solicitud de gran tamaño. Es común en sistemas de segmentación pura.

El papel del sistema operativo en la gestión de la memoria

El **sistema operativo** es el gestor de recursos y uno de los recursos más importantes que administra la memoria:

- Lleva el **registro** de la memoria: cuánto está libre y cuánto está asignado.
- Tiene políticas de **asignación de memoria**.
- Asigna memoria a los procesos cuando la solicitan.
- Libera la memoria cuando los procesos terminan.
- Administra el **sistema de memoria virtual**, permitiendo que los procesos usen más memoria de la físicamente disponible.

Espacio de direcciones virtuales del proceso

Cada proceso tiene un **espacio de direcciones virtual** dividido en regiones según su función:

- **Texto o código:** contiene el código ejecutable del programa.
- **Datos estáticos:** variables globales o estáticas con o sin inicialización.
- **Heap:** memoria dinámica asignada durante la ejecución.
- **Stack:** contiene los *frames* de pila con variables locales y direcciones de retorno de funciones.

Espacio de direcciones

¿Qué es el espacio de direcciones de un proceso?

El espacio de direcciones de un proceso es el conjunto de direcciones de memoria que dicho proceso puede acceder. Es un límite impuesto tanto por el hardware como por el sistema operativo para proteger y organizar el uso de la memoria.

¿Qué ocurre si un proceso accede a una dirección fuera de su espacio de direcciones?

No puede. Esa es, precisamente, la definición del espacio de direcciones: es el conjunto de direcciones a las que el proceso puede acceder.

¿Y qué ocurre si un proceso *intenta* acceder a una dirección fuera de su espacio de direcciones?

El hardware detecta esta acción ilegal, notifica al sistema operativo, y este normalmente termina el proceso con un error del tipo "*segmentation fault*".

¿Qué es el espacio de direcciones de usuario de un proceso?

Es el subconjunto del espacio de direcciones al que el proceso puede acceder cuando se encuentra en modo usuario (user mode), es decir, cuando no tiene privilegios de sistema.

¿Por qué el espacio de direcciones de un proceso no coincide con la memoria física de la máquina?

Por varias razones:

- En una misma máquina pueden ejecutarse múltiples procesos, cada uno con su espacio de direcciones.
- En los sistemas modernos, las direcciones que ve un proceso son lógicas o virtuales, no físicas.
- El espacio de direcciones puede ser mayor que la memoria física instalada.
- La estructura del espacio de direcciones permite la portabilidad del programa entre distintas máquinas.
- Algunas direcciones dentro del espacio de direcciones pueden estar reservadas para fines especiales.

¿Cómo está estructurado el espacio de direcciones de un proceso?

Está dividido en zonas (también llamadas segmentos o regiones), cada una con una función específica. Las zonas típicas incluyen:

- **Segmento de código:** instrucciones del programa.
- **Segmento de datos:** variables globales y estáticas.
- **Pila (stack):** para variables locales y llamadas a funciones.
- **Región para memoria dinámica (heap).**
- **Regiones mapeadas:** para archivos o memoria compartida.

¿Puede crecer el espacio de direcciones de un proceso?

Sí, puede crecer de varias formas:

- Al solicitar más memoria dinámica (por ejemplo, `malloc`), el segmento de datos o *heap* puede expandirse.
- Al mapear archivos, se crean nuevas regiones para el contenido del archivo.
- Al realizar llamadas recursivas o funciones con muchas variables locales, la pila puede crecer.
- Al usar memoria compartida, se crean nuevas regiones asignadas a esta memoria.

¿Puede reducirse el espacio de direcciones de un proceso?

Sí. Cuando un proceso libera memoria (por ejemplo, con `free` o al cerrar un *mapeo*), el espacio de direcciones puede disminuir y algunas regiones pueden liberarse completamente.

¿Existe algún comando para ver el espacio de direcciones de un proceso?

Sí. Algunos comandos disponibles en distintos sistemas operativos son:

- `pmap` en Linux y Solaris.
- `procstat -vm` en FreeBSD.
- `procmap` en OpenBSD.
- `vmmap` en macOS.

¿El espacio de direcciones de un proceso debe ser contiguo?

No. Puede tener "huecos" (holes). De hecho, esto es necesario para permitir el crecimiento de algunas regiones (por ejemplo, la pila o el *heap*) sin interferir con otras regiones. Estos huecos también ayudan a mantener una separación segura entre zonas de memoria.

Swap

¿Qué es el área de swap?

El área de **swap** es una parte de almacenamiento secundario utilizada como memoria auxiliar por el sistema operativo. Su función principal es permitir que el sistema soporte un mayor nivel de multiprogramación, es decir, más procesos en ejecución simultáneamente, aunque no todos quepan en la memoria principal.

¿Cuándo se utiliza el swap?

Un proceso en ejecución necesita estar en memoria. Si la RAM está llena, el sistema operativo puede **mover temporalmente procesos o partes de procesos al área de swap** para liberar espacio. Esto permite ejecutar otros procesos que sí necesitan estar en RAM en ese momento. Sin embargo, si un proceso que está en la zona de swap es seleccionado por el planificador,

debe **ser cargado de nuevo en memoria**, lo que incrementa el tiempo de cambio de contexto. En el caso de procesos que están esperando operaciones de entrada/salida, el sistema primero transfiere los datos a **buffers del núcleo**, y luego a los dispositivos de E/S, añadiendo más sobrecarga al sistema.

Swapping vs Paging

Swapping

- Consiste en mover **procesos completos** al área de swap.
- Se utilizaba en sistemas antiguos y ahora obsoletos.
- Permitía aumentar la multiprogramación con un alto coste de rendimiento.

Paging

- Se utiliza en los sistemas modernos con **memoria virtual**.
- En lugar de mover procesos completos, se transfieren **páginas individuales** del proceso.
- Ofrece una **ilusión de memoria casi ilimitada**, al gestionar de forma eficiente las necesidades de memoria de los procesos.

En resumen, hoy en día **los sistemas operativos modernos rara vez hacen swapping completo**, y en su lugar usan **paging**, que es mucho más eficiente.

Tipos de dispositivos de swap

El área de swap puede estar implementada como:

- Un **disco dedicado exclusivamente** al swap.
- Una **partición** específica dentro de un disco.
- Un **archivo dentro del sistema de archivos**.

Swap como archivo

- Es una **solución más flexible**: se puede cambiar fácilmente su tamaño o ubicación.
- Pero es **menos eficiente**, ya que implica acceder al swap a través del sistema de archivos, lo que añade capas de indirección.

Swap en diferentes sistemas operativos

- **MS Windows** utiliza un archivo de swap.
- **Sistemas tipo Unix** suelen usar **particiones de swap**, aunque también pueden configurarse para usar archivos si se desea más flexibilidad.

Reubicación y protección

¿Cuándo y dónde se asignan las direcciones de memoria?

1. Relocalización absoluta

- Las direcciones se determinan en tiempo de compilación y/o enlace.
- En ejecutable **sólo puede ejecutarse** en una posición específica de memoria.
- **No hay portabilidad**: no puede ejecutarse en otras ubicaciones.

2. Relocalización estática

- Las direcciones se calculan al **cargar** el programa en memoria.
- El ejecutable contiene **referencias relativas**.
- El procesador **no puede ser movido** a otra posición una vez cargado.
- El swapping sólo es posible si el proceso regresa a la misma ubicación.

3. Relocalización dinámica

- Las direcciones se determinan en **tiempo de ejecución**.
- El proceso trabaja con **direcciones lógicas o virtuales**, no físicas.
- Traducción realizada por **hardware especializado**.
- Permite **mayor flexibilidad**: los procesos pueden moverse libremente por la memoria.
- Permite **enlace dinámico**: bibliotecas compartidas.
- Requiere traducción de direcciones lógicas a físicas, **los sistemas modernos usan relocalización dinámica**

Protección de la memoria

La protección de memoria es esencial para garantizar la **estabilidad y seguridad** del sistema:

Reglas básicas de protección

- Un proceso **no debe acceder** directamente a la memoria del sistema operativo.
- Un proceso **no debe acceder** a la memoria de otros procesos.

Soporte hardware básico: dos registros de límites

- **Registro base**: dirección física mínima accesible por el proceso.
- **Registro límite**: tamaño del espacio lógico que puede utilizar el proceso.

Funcionamiento

- Cada dirección lógica generada por el proceso:
 - Debe ser **menor que el valor del registro límite**.
Luego se **suma al valor del registro base** para obtener la dirección física.
- Si la dirección lógica excede el límite, el hardware genera una **excepción**.

- Los valores de estos registros se actualizan en cada **cambio de contexto** y se almacenan en el **PCB** (Process Control Block).
- Cambiar estos valores es una **instrucción privilegiada**

Ventajas

- **Aislamiento de procesos.**
- El programa tiene la ilusión de ejecutarse en su propia máquina, con una memoria que comienza en 0.
- Este esquema corresponde a un **sistema segmentado con un único segmento.**

Protección moderna

- **Segmentación y paging** son las técnicas actuales que permiten:
 - Protección efectiva de la memoria.
 - Relocalización dinámica.
- Es necesario contar con al menos **dos modos de ejecución**:
 - **Modo usuario**: ejecución restringida.
 - **Modo kernel**: acceso completo, reservado al sistema operativo.

Esquemas simples de gestión de memoria

Sistemas sin multiplicación

Este enfoque ha estado **obsoleto durante décadas**. En estos sistemas:

- Sólo existían **dos zonas de memoria**:
 - Una para el **sistema operativo**.
 - Otra para el **proceso de usuario**.
- Disposición típica:
 - El SO en las posiciones **bajas de memoria**.
 - El resto asignado al proceso de usuario.

Sistemas con multiprogramación

Para los Sistemas Operativos multiprogramados, el esquema más simple consiste en **dividir la memoria en particiones**, cada una conteniendo un proceso. Hay 2 alternativas:

1. Partición de tamaño fijo
 - **Número fijo de procesos** en memoria.
 - Problemas:
 - **Fragmentación interna**

- **Fragmentación externa**

2. Particiones de tamaño variable

- **Tamaño y número de particiones pueden cambiar** dinamicamente.
- Venajas:
 - **Fragmentación interna despreciable**
- Inconvenientes:
 - **Fragmentación externa** sigue presente.
 - Se requería **compactación de memoria**, una operación **muy costosa**.

Segmentación (Obsoleta)

La **segmentación** fue una técnica de gestión de memoria que **reflejaba la forma en que los programadores estructuran los programas**: como una colección de segmentos lógicos.

- **Segmento**: unidad lógica como: programa principal, funciones, métodos, objetos, variables locales/globales, pila, tablas de inodos, arreglos, etc.
- Cada proceso tiene un **espacio de direcciones dividido en segmentos** de tamaño variable.

Direcciones en segmentación

Una **dirección lógica** se expresa como un par: <número de segmento, desplazamiento>.

El número de segmento apunta a una **entrada en la tabla de segmentos**:

- **Base**: dirección física inicial del segmento.
- **Límite**: tamaño del segmento.

Hardware y protección

- El registro **Segment Table Base Register, STBR**, señala la ubicación de la tabla de segmentos.
- Cambiar registros de segmento en un cambio de contexto es una **instrucción privilegiada**.
- La protección se implementa en cada entrada de la tabla:
 - **Bit de validación**
 - **Permisos**
 - **Modo de acceso**
- **Compartir código o datos a nivel de segmento**.

Fragmentación

- **Fragmentación interna:** Tamaños de segmento múltiplos de una unidad fija. Espacio desaprovechado **dentro del segmento**. En general, **despreciable**.
- **Fragmentación externa:** Los segmentos son de tamaño variable -> se crean **huecos de tamaños diversos** -> **Compactación**, resuelve el problema, pero con alto coste computacional.

Asignación dinámica del almacenamiento

- El SO gestiona bloques libres y ocupados. Al liberar memoria, **fusiona huecos adyacentes**.

Algoritmos de asignación:

- **First fit:** primer hueco que quepa. Rápido
- **Next fit:** desde el último hueco usado.
- **Best fit:** hueco más pequeño que sea suficiente.
- **Worst fit:** hueco más grande. Suele ser menos eficiente.

Se ha demostrado que *first fit* y *best fit* tienen mejor rendimiento que *worst fit*.

Estimación de memoria desaprovechada:

- Sean:
 - s : tamaño promedio de segmento.
 - k : tamaño promedio de hueco.
- Total de memoria desaprovechada respecto al total: $\frac{k}{k+2}$

Soporte en hardware

- Requiere soporte de hardware para:
 - Traducción dinámica de direcciones.
 - Protección de memoria.
 - Compartición de segmentos.
- No requiere que el espacio de direcciones lógico y físico tengan el mismo tamaño.

Paginación

La paginación es una técnica de gestión de memoria que permite que el espacio de direcciones físicas de un proceso sea no contiguo. Es decir, un proceso puede ser cargado en cualquier lugar disponible de la memoria física. Esto soluciona varios problemas importantes:

- **Evita la fragmentación externa**, aunque introduce fragmentación interna.
- **Elimina la necesidad de dividir la memoria en bloques de tamaño variable**.

- **Permite la reubicación dinámica, protección de memoria y compartición entre procesos.**

Estructura de paginación

- **Memoria física:** se divide en bloques de tamaño fijo llamados *frames*, cuyo tamaño es potencia de 2, típicamente entre 512 bytes y 16 MB.
- **Memoria lógica (virtual):** se divide en bloques del mismo tamaño llamados *pages*.
- Los espacios de direcciones lógicas y físicas no necesitan tener el mismo tamaño.

El sistema operativo mantiene una lista de *frames* libres. Para ejecutar un proceso que necesita N páginas, se buscan N *frames* libres donde cargar el programa. Para traducir direcciones lógicas a físicas, el sistema operativo configura una **page table**, cuya dirección base se almacena en un registro especial llamado **Page Table Base Register**. Este registro se actualiza en cada cambio de contexto, ya que es una instrucción privilegiada.

Traducción de Direcciones

La dirección generada por la CPU se divide en dos partes:

- **Número de página:** índice en la tabla de páginas.
- **Desplazamiento:** posición dentro de la página.

La entrada correspondiente en la tabla de páginas da el número de *frame* físico donde se encuentra la página lógica. La dirección física final es la combinación del *frame* base y el desplazamiento.

Características de Paging

- **No hay fragmentación externa.**
- **Fragmentación interna** crece con el tamaño de página.
- **Páginas pequeñas:** menor desperdicio, pero mayor sobrecarga por tablas de páginas grandes.
- El sistema operativo debe gestionar tanto los *frames* libres como las tablas de páginas activas.
- Cada proceso ve su propia memoria lógicas, totalmente distinta de la física.

Protección y Compartición

- Un proceso sólo puede acceder a su propia memoria a través de su tabla de páginas.
- La tabla sólo puede ser modificada por el sistema operativo.
- Es posible compartir memoria: dos procesos pueden tener entradas en sus tablas apuntando al mismo *frame* físico.

Soporte de Hardware

- Algunos procesadores antiguos no soportan paginación.
- Se requiere hardware de traducción, como **TLB (Translation Lookaside Buffer)**.

TLB: Memoria asociativa para traducción rápida

- Caché rápida que almacena pares <número de página lógicas, número de *frame* físico.
- Si hay acierto, se accede directamente al *frame* físico.
- Si hay fallo, se consulta la tabla de memoria y se actualiza la TLB.
- Algunas TLBs usan identificadores de espacio de direcciones para evitar limpiar la TLB en cada cambio de contexto.

Tablas de paginación invertidas

- En sistemas con espacio de direcciones muy grandes, las tablas de páginas pueden ser enormes.
- Soluciones:
 - **Paginación multinivel**
 - **Tablas de páginas invertidas**

En una **tabla de páginas invertida**, cada entrada corresponde a un **frame físico**, no a una página lógica. Contiene información del proceso y número de página que ocupa ese *frame*. Se usa una función hash para acelerar búsquedas. Utilizado en arquitecturas como **IBM PowerPC** y **AS/400**.

Sistemas mixtos

Los sistemas modernos de gestión de memoria utilizan combinaciones de técnicas de paginación y/o segmentación. Aunque existen variantes, la paginación siempre está como último mecanismo de gestión, ya que permite un uso más eficiente de la memoria.

Tipos de sistemas mixtos

- **Segmented Paging**: Obsoleto hoy en día. Fue utilizado en sistemas como el **IBM System/370**, donde la tabla de páginas de un proceso estaba segmentada.
- **Paged segmentation**: Variante donde los segmentos de memoria están paginados.
- **Multilevel Paging**: Se implementan varias capas de tablas de páginas para reducir el uso de memoria y optimizar el acceso.

Los sistemas actuales utilizan esquemas más sofisticados:

- **Arquitectura Intel x86 de 32 bits:** Paginación segmentada con **dos niveles** de tablas de páginas.
- **Arquitectura Intel x86 de 64 bits:** Utiliza cuatro niveles de paginación.

Ventajas de la paginación multinivel

A priori, la paginación multinivel es más compleja y puede ser más lenta que la paginación de un sólo nivel, aunque esto depende del rendimiento de la memoria, las cachés y la TLB.

Arquitecturas Intel de 32 y 64 bits

Arquitectura Intel de 32 bits

En los PC de 32 bits, se utiliza un sistema de **segmentación combinada con dos niveles de paginación**.

- **Segmentación:**
 - Cada dirección lógica se compone de un **selector** y un **desplazamiento**.
 - El selector contiene 13 bits para el índice del segmento, 1 bit para seleccionar la tabla y 2 bits de nivel de privilegio.
 - Se accede a una **tabla de descriptores**, de donde se obtiene una dirección base y un límite
- Cálculo de dirección lineal:
 - La dirección base del segmenteo se suma al desplazamiento, generando una **dirección lineal** de 32 bits.
- **Paginación en dos niveles:**
 - Los 10 bits más significativos de la dirección lineal indican una entrada en la **tabla de directorios de página**.
 - Los siguientes 10 bits se usan para indexar la **tabla de páginas** correspondiente.
 - Los últimos 12 bits indican el **desplazamiento dentro de la página**

Cada tabla de páginas tiene 1024 entradas, por lo que ocupa exactamente **una página**.

Arquitectura Intel de 64 bits

En la práctica, incluso los procesadores de 64 bits no utilizan todas las direcciones posibles. Actualmente se manejan:

- **48 bits de direcciones virtuales**
- **52 bits de direcciones físicas**

Direcciones virtuales válidas

- Los 16 bits más significativos deben ser todo 0s o todo 1s.
- Esto divide el espacio en dos mitades:
 - **Mitad inferior:** 0x0000000000000000 a 0x00007FFFFFFFFFFFFF
 - **Mitad superior:** 0xFFFF800000000000 a 0xFFFFFFFFFFFFFFFF
- Las direcciones intermedias no se consideran válidas.

Paginación de cuatro niveles

- **Tamaño de página:** 4 KB
- **12 bits:** desplazamiento dentro de la página.
- **32 bits:** número de página virtual, dividimos en 4 niveles de 9 bits cada uno
 - Cada nivel tiene 512 entradas, y cada entrada ocupa 8 bytes -> una tabla ocupa 4 KB
- El número de página virtual se consulta en la **TBL (Translation Lookaside Buffer)**
 - Si hay acierto, se obtiene el **número de página física (PPN)** de 40 bits.
- Se suma el desplazamiento -> Dirección física de 52 bits.
- Esta dirección física se usa para acceder a la **caché L1**, con:
 - 40 bits para la etiqueta (tag).
 - 6 bits de índice
 - 6 bits de desplazamiento

Introducción a memoria Virtual

¿Dónde están las páginas?

Esto plantea otra pregunta: **Si una página no está en memoria, ¿dónde está? ¿De dónde la saca el sistema operativo?**

Depende de si la página es de **código, datos o pila (stack)** y de si ha sido **modificada**:

- **Código:** Páginas de solo lectura, se cargan desde el archivo ejecutable.
- **Datos:**
 - Si están **inicializados**, se cargan desde el archivo ejecutable.
 - Si **no tienen valores iniciales**, se asignan nuevas páginas y se **rellenan con ceros** (por eso en C las variables externas y estáticas sin inicializar se ponen a 0).
 - Si fueron **modificadas**, se toman del **dispositivo de intercambio (swap)**.
- **Pila (stack):**
 - Las páginas nuevas no modificadas se **asignan y se llenan con ceros**.
 - Las ya usadas y modificadas se toman del **dispositivo de intercambio**.

Por tanto, para tener **memoria virtual**, se necesita un **dispositivo de almacenamiento auxiliar llamado "dispositivo de intercambio"** (swap) para guardar las páginas modificadas

de datos y pila.

¿Qué se necesita para tener memoria virtual?

La implementación más común es la **paginación por demanda**, y para que funcione se requiere:

1. **Un esquema de direccionamiento con paginación y bit de presencia**, para que al referenciar una página con bit 0, la MMU genere una excepción.
2. **Un dispositivo de almacenamiento para guardar páginas**, llamado **dispositivo de intercambio** (swap). Puede ser:
 - **Un archivo** (como en Windows, más flexible).
 - **Una partición de disco** (como en Unix, más rápida al evitar la sobrecarga del sistema de archivos).
3. **Un sistema operativo capaz de gestionar esa excepción**:
 - Guarda el estado del proceso.
 - Si la dirección es ilegal → actúa en consecuencia (normalmente termina el proceso).
 - Si es válida pero la página no está en memoria → localiza la página en el swap, la carga, actualiza la tabla de páginas y reanuda el proceso.

La memoria virtual no es tan simple

En la explicación anterior, la memoria virtual parece simple, pero hay **muchos detalles adicionales**. Por ejemplo, en la instrucción "leer la página en memoria" hay varias consideraciones:

- ¿**Planificamos otro proceso** mientras se espera al dispositivo?
- ¿**Tenemos un *frame* libre** en memoria?
- Si no hay marco libre, habrá que **reemplazar una página** existente:
 - ¿Debe ser una página del mismo proceso? (esto se llama **ámbito de reemplazo**).
 - ¿Cómo decidir qué página reemplazar? (esto es la **política de limpieza**).
 - ¿Cargamos una sola página o intentamos anticipar y cargar varias? (**política de precarga** o *fetching*).

Segmentos de software

Segmentación vs paginación

Ya hemos visto qué eran la segmentación y la paginación a nivel de hardware. Incluso hemos visto ejemplos reales de segmentación paginada, como en la arquitectura de PC de 32 bits de Intel.

Por un lado, la segmentación se acerca más a la estructura lógica de un programa, ya que

permite separar regiones como el código, los datos, el heap, las bibliotecas, la pila, etc. Por otro lado, la paginación es mucho más eficiente en cuanto a memoria, ya que todas las páginas tienen el mismo tamaño. Cada paginación puede ser reemplazada por otra. Por eso, la mayoría de los sistemas operativos implementan la memoria virtual mediante **paginación bajo demanda**. Aún así, existieron algunos sistemas operativos que no lo hicieron, como el OS/2 1.x de IBM, que utilizaban segmentación bajo demanda. En la actualidad, la mayoría del hardware sólo proporciona paginación, y aunque hay hardware que también ofrece segmentación, esta rara vez se utiliza.

¿Qué es un segmento de software?

Pensemos en un ejemplo muy simplificado.

Supongamos un sistema paginado de 16 bits: 6 bits para el número de página y 10 para el desplazamiento dentro de la página.

Ahora consideramos un proceso muy simple que tiene:

- Código en las páginas 0, 1 y 2
- Datos en las páginas 4, 5, 6, 7 y 8
- Pila en las páginas 38 y 39

Traducido a direcciones lógicas, el proceso tendría:

- Código que comienza en la dirección 0x0000, con un tamaño de 3 KB (Hasta 0x0BFF)
- Datos que comienzan en 0x1000, con un tamaño de 5 KB (Hasta 0x23FF)
- Pila que comienza en 0xE000, con un tamaño de 2 KB (Hasta 0xE7FF)

Imaginemos ahora que el sistema operativo mantiene el espacio de direcciones del proceso como una lista de elementos, cada uno describiendo una "zona" distinta del espacio de direcciones.

Cada elemento tiene la dirección virtual de inicio, el tamaño y los permisos.

Fallos de página y errores de segmentación

Imaginemos que todas las páginas del proceso, excepto la página 5 y la página 6, están en memoria. Esto significa que los bits de presencia para las páginas 0, 1, 2, 4, 7, 8, 38 y 39 están en 1, y el resto en 0.

- Si el proceso accede a la dirección 0x124A, que está en la página 4, la MMU la traducirá a una dirección física normalmente.
- Si accede a 0x14F2 (página 5), como su bit de presencia es 0, la MMU generará una **excepción**. El sistema operativo tomará el control y comprobará que esa dirección está dentro del rango 0x1000 a 0x23FF, por tanto es válida. Se trata de un **fallo de página**, y el sistema operativo lo atenderá.

- Si accede a la dirección 0x4F06 (página 19), su bit de presencia también es 0, y la MMU lanzará una excepción. Sin embargo, el sistema operativo comprobará que esa dirección **no pertenece a ningún segmento** definido. Esto genera un **error de segmentación**, lo que normalmente resulta en la finalización del proceso.

Esto demuestra que podemos tener "segmentos" de software aunque no haya segmentación por hardware. Linux llama a estas entidades **segmentos**, mientras que otros sistemas operativos las llaman **regiones**. El comando `pmap` muestra los segmentos de un proceso, incluyendo su dirección de inicio, tamaño y permisos.

Tipos de segmentos

Existen dos tipos principales de segmentos:

1. **Segmentos vnode**: Están asociados a un archivo, como los segmentos de código o archivos mapeados. Las páginas de estos segmentos obtienen sus valores iniciales del archivo al que están asociados.
2. **Segmentos anónimos**: No están asociados a ningún archivo. Ejemplos de estos son los segmentos de datos y pila. Se asocian finalmente al dispositivo de swap. Las páginas de estos segmentos se inicializan con ceros cuando se produce un fallo de página por primera vez.

En Linux y Solaris, el comando `pmap` muestra los segmentos del espacio de direcciones de un proceso, junto con su dirección de inicio y tamaño, y en el caso de segmentos vnode, el archivo asociado.

Copy-on-Write

Existen dos situaciones en las que los segmentos deben duplicarse, por ejemplo durante la llamada al sistema `fork()`.

- Si los segmentos son de sólo lectura, simplemente se **comparten**.
- Si son de escritura, se aplica un procedimiento llamado **Copy-on-Write**:
 - El segmento se comparte pero se marca como de sólo lectura.
 - Si se intenta escribir en ese segmento, se produce una excepción.
 - El sistema operativo toma el control y comprueba que la excepción se ha producido al intentar modificar un segmento originalmente de escritura pero ahora marcado como sólo lectura.
 - Entonces, **duplica la página** que causó la excepción, actualiza la tabla de páginas de uno de los procesos, y el acceso puede continuar.

Este mecanismo evita duplicar todo el segmento; **sólo se duplican las páginas realmente modificadas**

¿Qué es la memoria virtual y por qué funciona?

Memoria Virtual

La **memoria virtual** es una técnica que permite ejecutar programas que no están completamente cargados en la memoria física. Esto es posible porque, en la práctica, no todas las partes de un programa se utilizan durante su ejecución. Por ejemplo, algunos errores raros, funciones opcionales que nunca se llaman o variables sobredimensionadas pueden no llegar a utilizarse.

¿Por qué funciona la memoria virtual? Principio de localidad

El **principio de localidad** es la razón principal por la que la memoria virtual funciona. Este principio establece que, durante la ejecución de un programa, las referencias a memoria tienden a agruparse tanto en el proceso como en el tiempo:

- **Localidad temporal:** direcciones que se han usado recientemente tienden a usarse de nuevo pronto.
- **Localidad espacial:** si se accede a una dirección de memoria, es probable que pronto se acceda a direcciones cercanas.

El sistema operativo puede aprovechar esto asignando memoria sólo a las porciones activamente utilizadas del programa, reduciendo la cantidad de memoria física necesaria.

Ventajas de la memoria virtual

El uso de memoria virtual ofrece varias ventajas importantes:

- Permite ejecutar programas más grandes que la memoria RAM instalada.
- Aumenta el grado de multiprogramación, ya que varios procesos pueden compartir la RAM.
- Reduce el uso de entrada/salida durante el intercambio de programas completos, ya que sólo se intercambian las páginas necesarias.

Implementación: Paginación bajo demanda

La forma más común de implementar memoria virtual es mediante **paginación bajo demanda**. En esta técnica, las principales páginas del programa se cargan a la memoria sólo cuando son necesarias.

Este método permite ejecutar programas que superan el tamaño de la memoria física, a costa de una cierta pérdida de rendimiento. Esto introduce la necesidad de decidir **qué página reemplazar** cuando no hay *frames* libres, y si hacerlo con un algoritmo de reemplazo **local** (dentro del proceso) o **global** (compartido entre procesos).

También es clave definir **cuántos frames se asignan a cada proceso** para optimizar el rendimiento y minimizar fallos de página.

Desempeño de la paginación bajo demanda

El servicio de un fallo de página incluye:

1. **Atención del fallo de página:** el sistema operativo toma el control, guarda el estado del proceso y localiza la página en el dispositivo de intercambio.
2. **Transferencia de la página:** espera en la cola del dispositivo, realiza la búsqueda, espera la latencia, transfiere la página y actualiza la tabla de páginas.
3. **Reanudación del proceso:** se restaura el estado del proceso y se continúa su ejecución.

Thrashing

El **thrashing** ocurre cuando el sistema pasa más tiempo manejando fallos de página que ejecutando procesos útiles. Esto sucede cuando:

- Un proceso no tiene suficientes *frames* asignados para su localidad activa.
- Cada nuevo acceso reemplaza una página que aún es necesaria, generando un ciclo de fallos constantes.

Si se usa reemplazo **global**, este problema puede propagarse a otros procesos, afectando al sistema completo.

Prevención del thrashing

Para evitar el thrashing, el sistema operativo debe asignar suficientes *frames* a cada proceso. Hay dos estrategias principales para decidir esto:

- **Modelo del conjunto de trabajo:** intenta identificar cuántas páginas usa activamente un proceso.
- **Modelo de frecuencia de fallos de página:** ajusta los *frames* asignados en función de la tasa de fallos del proceso.

Ubicación, carga y reemplazo de páginas en memoria virtual

Consideraciones generales

El diseño de un sistema de memoria virtual implica tomar diversas decisiones fundamentales. Cada una de ellas afecta directamente al rendimiento, eficiencia y complejidad del sistema. Entre las principales cuestiones a resolver se encuentran:

- ¿Dónde se coloca en memoria la página recientemente referenciada? -> **Política de ubicación de páginas**

- ¿Sólo se carga la página que causó el fallo? -> **Política de carga de páginas**
- ¿La página que se reemplaza pertenece al mismo proceso que causó el fallo? -> **Ámbito de reemplazo**
- ¿Qué criterio se utiliza para elegir la página a reemplazar? -> **Algoritmo de reemplazo**

Política de ubicación de páginas

La política de ubicación de páginas define dónde se colocará en la memoria física una página cuando es traída desde el almacenamiento secundario.

- En sistemas **paginados**, este problema no es relevante, ya que todos los *frames* son iguales.
- En sistemas **segmentados**, sí es una precaución importante.
- En sistemas de **segmentación paginada**, la memoria también se asigna por páginas, por lo que tampoco suele representar un problema.
- En sistemas **NUMA**, sí es un aspecto crítico debido a las diferencias de latencia entre nodos de memoria.

Política de carga de páginas

Cuando ocurre un fallo de página, el sistema operativo puede optar por diferentes estrategias de carga:

- **Paginación por demanda pura:** sólo se carga la página que fue referenciada. Esto implica que el sistema no carga ninguna página de un proceso hasta que esta sea usada.
- **Prefetching:** Se carga la página requerida y algunas páginas adicionales. Esta técnica aprovecha que muchos dispositivos de swap funcionan más eficientemente al transferir varias páginas de una vez. Si embargo, puede introducir ineficiencias si las páginas adicionales no llegan a utilizarse.

Comparación:

- La paginación por demanda produce muchos fallos iniciales, pero con el tiempo disminuyen al cargarse más páginas.
- El prefetching puede reducir fallos, pero con el riesgo de cargar páginas innecesarias.

Reemplazo de páginas

La política de reemplazo determina qué página será descartada de la memoria física cuando sea necesario hacer espacio para una nueva. Este proceso involucra tres aspectos fundamentales:

1. **Asignación de *frames*:** ¿Cuántos **frames** se asignan a cada proceso?

2. **Ámbito del reemplazo:** ¿Puede una página de un proceso reemplazar a otra de un proceso diferente?
3. **Algoritmo de reemplazo:** ¿Qué criterio se usa para seleccionar la página a eliminar?

Asignación de *frames*

Existen dos enfoques principales:

- **Asignación fija:** a cada proceso se le asigna un número fijo de *frames*.
- **Asignación variable:** el número de *frames* asignados a un proceso varía según sus necesidades durante la ejecución.

En la asignación fija, surge la cuestión de cómo determinar cuántos *frames* asignar:

- Mismo número para todos los procesos.
- Proporcional al tamaño del proceso.
- Según prioridad del proceso, tiempo de CPU, etc.

En la asignación variable, el sistema operativo intenta estimar la cantidad óptima de *frames* para cada proceso en cada momento.

- Si se asignan **menos *frames* de los necesarios**, se incrementan los fallos de página.
- Si se asignan **demasiados *frames***, se desaprovecha memoria.

Métodos de estimación

- **Modelo de conjunto de trabajo (Working Set Model)**
- **Algoritmo de frecuencia de fallos de página (Page Fault Frequency)**

Ámbito del reemplazo

Define si un proceso puede reemplazar sólo sus propias páginas o también las de otros procesos:

- **Reemplazo local:** sólo puede reemplazar páginas de otros procesos.
- **Reemplazo global:** puede reemplazar páginas de otros procesos.

Combinaciones comunes en sistemas modernos

Muchos sistemas operativos actuales como Linux o BSD Utilizan:

- **Asignación variable de frames**
- **Reemplazo global**

Esta combinación es fácil de implementar y permite aprovechar un **pool de frames libres** para cargar nuevas páginas sin demoras.

Caché de páginas

Los sistemas suelen mantener un **pool de frames libres** o **caché de páginas**.

Cuando se necesita reemplazar una página:

1. Se carga la nueva página en un *frame* de pool.
2. La página reemplazada se marca como libre y se añade nuevamente al pool.
3. Si esa página vuelve a ser referenciada y aún está en la caché, puede reutilizarse sin necesidad de ir al disco -> mejora el rendimiento.

Listas en la caché de páginas

- **Free Page List:** páginas robadas a los procesos, no modificadas.
- **Modified Page List:** páginas modificadas, deben escribirse al disco antes de liberarse.

Bloque de frames (*Frame locking*)

El sistema operativo puede proteger ciertos *frames* mediante un **bit de bloqueo**, lo que impide que las páginas que contienen sean reemplazadas.

Este mecanismo se utiliza para:

- Páginas del **kernel**.
- Estructuras críticas del sistema.
- Páginas involucradas en operaciones de E/S.

Generalmente no se aplica a páginas de usuario, ya que la E/S se realiza a través de **buffers del sistema** que sí están protegidos.

Algoritmos de reemplazo de páginas

Cuando se produce un fallo de página, ya sea en un esquema de reemplazo **local** o **global**, el sistema debe decidir **qué página residente se reemplazará**. Esta decisión está determinada por lo que se conoce como **algoritmo de reemplazo de páginas**.

Alcance del reemplazo

- **Reemplazo local:** sólo se consideran las páginas del proceso que provocó el fallo.
- **Reemplazo global:** se consideran las páginas de todos los procesos.

Los algoritmos descritos a continuación pueden aplicarse a ambos contextos.

Algoritmos clásicos de reemplazo

1. **Óptimo**
2. **LRU** (Leas Recently Used)
3. **FIFO** (First-In First-Out)

De estos tres, sólo **FIFO** puede implementarse fácilmente.

El óptimo no es viable en la práctica, ya que requiere conocimiento futuro, y **LRU** necesita hardware especial para una implementación eficiente.

1. Optimal Algorithm

- Reemplaza la página que **tardará más en volver a ser referenciada**.
- Produce el **mínimo número posible de fallos de página**.
- No es implementable en la práctica, ya que implica conocer el futuro de las referencias.
- Se utiliza como **referencia teórica** para comprar otros algoritmos.

2. LRU

- Reemplaza la página que **no ha sido usada recientemente**.
- Equivalente al algoritmo óptimo, pero considerando el pasado en lugar del futuro.
- Se basa en el **principio de localidad temporal**: las páginas recientemente usadas tienden a ser reutilizadas pronto.

Implementaciones

- **Contadores**: cada página almacena el momento exacto en el que fue usada.
- **Listas**: las páginas se reorganizan en una lista según su uso reciente.

3. FIFO

- Reemplaza la **página más antigua en memoria**.
- Es fácil de implementar con una cola circular.
- **Problema**: sufre la **anomalía de Belady**, donde **más frames no siempre implican menos fallos**.

Stack Algorithms

Un **algoritmo de pila** garantiza que el conjunto de páginas residentes con N frames es **subconjunto** del que se tendría con $N+1$.

Los algoritmos óptimo y LRU son de pila, por lo que **no sufren la anomalía de Belady**.

Algoritmos de reemplazo aproximados

Dado que **LRU no puede implementarse directamente**, se utilizan algoritmos aproximados, aprovechando soporte de hardware que suele proporcionar:

- **Bit de referencia**
- **Bit de modificación**

Second chance algorithm

- Extiende FIFO utilizando el **bit de referencia**.
- Estructura: cola circular
- Cuando se debe reemplazar:
 1. Si el bit de referencia de la página apuntada es 0, se reemplaza.
 2. Si es 1, se limpia y se da una "**segunda oportunidad**", avanzando al siguiente *frame*.
- El **puntero que recorre la cola** funciona como una manecilla de reloj -> también llamado **Clock Algorithm**.

Variantes del Clock Algorithm

1. ****Con bits (referencia, modificación)**
 1. Se busca una página con bits (0,0) para reemplazar.
 2. Si no se encuentra, se busca una con (0, 1), limpiando los bits de referencia la pasar.
 3. Si no se encuentra, se repite el proceso
2. **Doble índice**
 - Se utilizan **dos punteros**:
 - El primero limpia los bits de referencia.
 - El segundo identifica páginas que no fueron referenciadas desde el último pase.
 - Se ejecuta **periódicamente**.
 - Utilizando, por ejemplo el **algoritmo de page stealing en UNIX**.

Uso de bits de referencia adicionales

- El sistema operativo mantiene un **contador por página**.
- Periódicamente:
 - Lee el bit de referencia.
 - Inserta su valor a la izquierda del contador y desplaza a la derecha los anteriores.
- El **valor más bajo** representa la página menos recientemente usada.

Algoritmo	Implementación	Calidad	Anomalía de Belady
Optimal	No posible	Excelente	No

Algoritmo	Implementación	Calidad	Anomalía de Belady
LRU	Difícil	Muy buena	No
FIFO	Muy fácil	Regular	Sí
Second Chance	Fácil	Buena	Depende
Clock Variantes	Moderada	Buena	No

Adaptación del tamaño del conjunto de residentes: alcance y asignación combinados.

La gestión del conjunto residente de un proceso implica decisiones sobre **el alcance de reemplazo y la asignación de *frames***. Al combinar ambas dimensiones, se derivan cuatro escenarios posibles:

Alcance de reemplazo:

- **Reemplazo global:** las páginas reemplazadas pueden pertenecer a cualquier proceso.
- **Reemplazo local:** sólo se reemplazan páginas del mismo proceso que provocó el fallo.

Asignación de *frames*:

- **Asignación fija:** el número de *frames* asignados del mismo proceso que provocó el fallo.
- **Reemplazo local:** el número de *frames* asignados a un proceso permanece constante.

Combinaciones posibles:

1. Reemplazo global con asignación fija
2. Reemplazo global con asignación variable
3. Reemplazo local con asignación variable
4. Reemplazo local con asignación fija

La opción 1 no es viable, ya que implica contradicción: si la asignación es fija, no tiene sentido permitir que el reemplazo sea global. Las otras combinaciones son posibles y presentan características particulares.

1. Reemplazo global con asignación variable

Es el enfoque adoptado por la mayoría de los sistemas operativos modernos como Linux, FreeBSD o Solaris.

- **Ventajas:** fácil de implementar y gestionar.

- Cuando un proceso genera un fallo de página, se le asigna un nuevo marco, y la página a reemplazar puede pertenecer a cualquier proceso.
- Se mantiene un **pool de frames libres**, y un proceso "page stealer"

2. Reemplazo local con asignación fija

- Cada proceso mantiene un conjunto residente constante durante su ejecución.
- **Desventajas:**
 - Si se asignan pocos *frames*: muchos fallos de página.
 - Si se asignan demasiados: desperdicio de memoria y baja multiprogramación.
- **No se usa en sistemas modernos** debido a su rigidez.

3. Reemplazo local con asignación variable

- El sistema ajusta dinámicamente el número de *frames* asignados al proceso según su comportamiento.
- Cuando ocurre un fallo de página:
 - Se reemplaza una página del mismo proceso.
 - El sistema evalúa si el conjunto residente actual es adecuado.
 - Si hay muchos fallos: se **incrementa** el número de *frames*.
 - Si hay pocos fallos: se **reduce** el número de *frames*.
- Para decidir estos ajustes, se pueden usar dos modelos:

Modelos para adaptar el conjunto residente

1. Modelo de conjunto de trabajo

- Basado en el principio de **localidad temporal**.
- El **Conjunto de trabajo** $w(t, \Delta)$ es el conjunto de páginas referenciadas entre los instantes $t - \Delta$ y t .
- Si Δ es:
 - Muy pequeño: no captura toda la localidad.
 - Muy grande: mezcla varias localidades.
- El sistema intenta asignar a cada proceso el número de *frames* necesarios para contener su conjunto de trabajo actual.
- **Si no hay suficiente memoria** para alojar el conjunto de trabajo de un proceso, se **suspende su ejecución**.

Diferencia clave con LRU:

- $W(t, \Delta)$ = páginas **referenciadas** en las últimas Δ referenciadas.

- LRU con Δ frames = últimas Δ páginas cargaas.

****Implementación aproximada:**

- Se puede usar un temporizador y un **bit de referencia** por página.
- Cada cierto número de referencias, los bits se reinician.
- Una página con bit activo se considera parte del conjunto de trabajo.

2. Frecuencia de fallos de página

- Método directo y eficiente para evitar el **thrashing**.
- Se establece un **límite de tiempo o referencias** entre fallos de página.
 - Si el tiempo entre fallos es **menor** que el límite, se añade un *frame*
 - Si es **mayor**, se **eliminan** páginas no referenciadas desde el último fallo.

Versión con doble umbral

Si el intervalo es:

- **Menor** al umbral inferior -> se **añade** un *frame*.
- **Mayor** al umbral superior -> se **eliminan** páginas.
- **Entre ambos** -> se hace un reemplazo local normal.

Comparación y conclusión

Modelo	Método	Reacción a PFF alto	Reacción a PFF bajo
Working Set	Basado en historial de acceso	Suspende si no hay memoria	Ajusta el conjunto de trabajo
Page-Fault Frequency	Basado en tiempo entre fallos	Asigna más marcos	Libera marcos

Segmentación bajo demanda

Cuando no se dispone de hardware de paginación, es posible implementar **memoria virtual mediante segmentación bajo demanda**. Para ello se requiere:

- Un **dispositivo de swap**.
- Hardware de segmentación que incluya un **bit de presencia**, el cual indica si un segmento está cargado en memoria.

El funcionamiento básico es el siguiente:

- Si un **segmento está en memoria** y se referencia, se accede de forma normal.
- Si un **segmento no está en memoria**, se lanza una **excepción** que provoca su carga desde el dispositivo de swap.

Algoritmo de reemplazo

Los algoritmos de reemplazo en segmentación bajo demanda son similares a los utilizados en paginación bajo demanda, aunque adaptados al tamaño variable de los segmentos.

En el caso de OS/2, el sistema mantenía una **lista de segmentos residentes en memoria**. De forma periódica:

- Los segmentos accedidos se movían al **final de la lista**.
- Se limpiaban los **bits de acceso**.

Cuando era necesario reemplazar un segmento, se elegía el **primer segmento** de la lista, lo que constituye una **aproximación a LRU**, ya que la lista se mantiene ordenada por **tiempo de último acceso**.

Segment replacement en segmentación bajo demanda

El mecanismo de reemplazo difiere del de paginación debido a que los segmentos **no tienen tamaño uniforme**. El procedimiento que seguía OS/2 ante un fallo de segmento era:

1. Se comprueba si hay suficiente espacio libre en memoria.
 - Si lo hay, se realiza una **compactación** para liberar espacio contiguo.
2. Si no hay espacio suficiente:
 - Se toma el **primer segmento** de la lista.
 - Si es necesario, se guarda en el **dispositivo de swap**.
3. Si ahora hay espacio, se carga el nuevo segmento en memoria, se actualiza la **tabla de segmentos** y se coloca el segmento cargado al final de la lista.
4. Si todavía no hay espacio suficiente, se **vuelve al paso 1**.

Otras consideraciones

Además del algoritmo de reemplazo, influyen en el rendimiento:

- El tipo de reemplazo: **local** o **global** uso.
- El tipo de asignación de *frames*: **fija** o **variable**.
- El **tamaño de página**:
 - **Páginas pequeñas**: mejor adaptación a la **localidad**.
 - **Páginas grandes**: simplifican el seguimiento y optimizan las **transferencias de disco**.

Colocación de datos en memoria

Aunque la paginación bajo demanda es **transparente** para el usuario y el programador, ciertos patrones pueden afectar significativamente al rendimiento.

En **lenguajes como C**, donde las matrices se almacenan **por filas**, recorrerlas **por filas** o **por columnas** puede tener efectos drásticos en la cantidad de **fallos de página**.

1. [Introducción a sistemas operativos](#)

2. [Ficheros](#)

3. [Memoria](#)

4. [Procesos](#)

5. [Entrada Salida](#)